

Voxel-Addressable Memory: A Unified Spatial Index for Sovereign AI

Authors: Weslyn Whitehead Jr.¹

Affiliations:

¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: <https://orcid.org/0009-0005-7707-3210>

Submitted: June 2026

Working Paper Series: AAMT-WP-20

Anchor DOI: 10.5281/zenodo.19600795

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Status: Working paper, not peer reviewed. All results were reproduced on an Apple M1 Max (32 GB) against the platform’s own corpora. The address hierarchy, rolling-memory loop, whitening, consolidation, and HNSW index live in the MaiaM Alchemist engine (`apps/desktop/scripts/voxel.py`, `sidecar-placeholder.py`, `embedder_bridge.py`, `memory_index_bridge.py`); the separation probe, recall harness, and IVF/HNSW scaling sweeps are standalone, embedding-cached scripts available on request. This paper extends **WP-15 (Vortex-Addressed Semantic Memory)** — which retrieves by TERA coordinate alone — with a learned sub-voxel residual and a navigable small-world index, and reports the measured retrieval and scaling results WP-15 left open.

Abstract

We present a memory architecture for long-context language models built on a single idea borrowed from voxel game engines: take a continuous space, partition it into cells, store only the occupied cells, and look them up without scanning everything. We show that the AAMT routing stack already encodes such a partition (the 4-bit Meji and 8-bit Odu lattices), formalize it as a strict coarse-to-fine voxel hierarchy over the TERA coordinate cube, and extend it with a learned sub-voxel residual indexed by a navigable small-world graph (HNSW). The result is a two-tier rolling memory whose effective context grows from a fixed working set to a million-item global store at constant attention cost. We report a 64% relative gain in retrieval recall from a stack of cheap, composable levers, a 285x scan-cost reduction at full recall retention on a 1M-item store, and an honest negative result on off-the-shelf rerankers. We close with the cross-stack reach of the same primitive.

1. The problem is not “tokens”

A transformer’s context limit is two physical walls, and they scale differently. The **KV cache** costs $2(k,v) \times \text{layers} \times \text{kv_heads} \times \text{head_dim} \times \text{dtype}$ bytes *per token retained*, a linear wall. **Attention** is $O(n^2)$ pairwise, a quadratic wall. “Context length in tokens” is just the unit these scale in. To reach for more usable context you must hold the *information* without holding the *KV entries* or paying the n^2 pairs.

Information theory forbids the naive escape: arbitrary text cannot be losslessly re-encoded 10-100x smaller. So the win is never “same information, fewer tokens.” It is “the right information retrieved on demand instead of all information held resident.” That reframes the design as a **retrieval** problem over a **space of memories**, which is where spatial-indexing technique applies.

2. The latent voxel grid in TERA space

AAMT routing already projects text to a TERA coordinate [T, E, R, A] (Temporal, Emotional, Rational, Archetypal) in the unit 4-cube. Two existing constructs quantize that cube:

- **Meji** — each axis binarized at 0.5 gives a 4-bit word, $\text{idx} = (\text{T} \ll 3) \mid (\text{E} \ll 2) \mid (\text{R} \ll 1) \mid \text{A}$, indexing 16 cells. This is the first-order router’s expert selector (`_resolve_meji_expert`). It is a uniform voxel grid: 1 bit per axis, $2^4 = 16$ cells.
- **Odu** — the ordered Meji pair (`prev -> curr`) gives an 8-bit word, 256 cells, used by the second-order transition router (`12-odu-second-order-routing`). $256 = 2^8$ is a finer grid; an Odu cell is a Morton-style composite of two 4-bit coordinates.

The stack was voxelizing meaning-space without naming it. We formalize this as a single hierarchy.

2.1 A strict coarse-to-fine hierarchy

Quantize each TERA axis to 2 bits (quartiles), packed T,E,R,A high to low, for a static 8-bit **Odu cell** (`voxel.tera_to_odu`). The high bit of each axis’s 2-bit level is exactly that axis’s **Meji bit** (the 0.5 threshold). Therefore:

Meji is recoverable from Odu by extracting the high bit of each axis. Every Meji cell contains exactly 16 Odu children. The result is a hexadeca-tree (a 4-D octree) over the TERA cube.

Verified empirically over 10,000 random coordinates: refinement holds for all, 16 Meji cells, exactly 16 Odu children each. A **Morton key** (`meji << 8`) `| odu` packs both levels into one sortable integer, so coordinates close in TERA produce close keys, the cache-friendly Z-order trick from voxel rendering. The canonical implementation is `apps/desktop/scripts/voxel.py`.

2.2 The third level: the residual sub-voxel

TERA is only 4 dimensions, far too coarse to *address* millions of distinct memories (we measured its standalone retrieval recall@10 at ~0.02, near useless). The fine address is a **learned residual**: the full sentence embedding, whitened and indexed. The complete address of a memory is therefore a product code:

L1	Meji	1 bit/axis	16 cells	coarse bucket / octree root
L2	Odu	2 bits/axis	256 cells	fine bucket; Meji = its high bits
L3	residual	whitened emb	HNSW graph	sub-voxel detail, the real discriminator

This is the same structure production vector databases use (FAISS IVF-PQ): coarse learned cells plus a quantized residual. We arrived at it from the gamedev door and met the state of the art.

3. Architecture

Two tiers. Tier 0 is the literal KV-cache working set (small, lossless, fast). Tier 1 is the geometric store: every span evicted from the window is encoded to its product-code address and folded into Tier 1. Each turn, the current context is projected, the nearest stored spans are retrieved, and a few rehydrated “memory tokens” are injected into the working set. Effective context becomes `working_set + entire_store` at constant attention cost.

Consolidation (greedy meshing). On encode, any same-session span within cosine 0.95 of an existing attractor is *merged* into it via a running-mean centroid rather than stored as a near-

duplicate. This is the associative-memory analog of merging adjacent identical voxels into one quad. It is also literal Hopfield consolidation; the store holds distinct attractors, not copies.

Global index (sparse chunk addressing). A persistent HNSW graph over the whitened residuals serves cross-session retrieval, the Vault tier. It lazy-builds from the store, adds incrementally on encode, and persists to disk with a store-fingerprint so cold starts load instead of rebuilding. The per-session hot path stays a small flat scan; HNSW is specifically the global-store engine.

Index-to-text rehydration. We store the raw text alongside the address and fetch exact bytes on a hit, rather than decoding content from a latent. For anything that must be verbatim this is strictly more robust than lossy latent reconstruction.

4. Measured results

All on an M1 Max / 32 GB against the platform’s natural-language corpora (**harmonic_reasoning**, **archetype_voice**, **asamanthinks_platform**) with code corpora as distractors. Ground truth is prompt->completion pairs: the query is a prompt, the correct memory is its own completion (same topic, different wording, a genuine paraphrase test).

4.1 Separation: where the ceiling lives

Embedding 1,232 natural-language spans with **all-MiniLM-L6-v2**: random pairwise cosine median 0.14 (well separated), but **nearest-neighbour cosine median 0.835**, with 33% of spans having a twin above 0.9, and **effective dimensionality ~30 of 384**. The bet’s crux is angular separation at scale, and MiniLM clusters hard. The address function, not the geometry, is the ceiling.

4.2 The recall stack (2,261-item store)

configuration	recall@1	recall@10
MiniLM-384, raw cosine (baseline)	0.074	0.202
+ whitening + consolidation	0.078	0.250
bge-large-1024, raw	0.095	0.238
bge + whitening + consolidation	0.117	0.331

A **+64% relative** gain in recall@10 and **+58%** in recall@1 from composable, cheap levers. The 4-bit TERA address alone scored recall@10 = 0.017, confirming the residual does the discriminating work. Whitening helped *more* on the high-dimensional embedder (raw->whiten +27% on bge vs +11% on MiniLM): more real structure to decorrelate.

4.3 An honest negative: rerankers

Adding an off-the-shelf MS-MARCO cross-encoder reranker **hurt** recall@10 (0.302 -> 0.203). It is trained for web-search query->passage relevance, not chat prompt->completion, and confidently misranks. We did not ship it. A useful reranker here requires domain-matched fine-tuning, not a drop-in.

4.4 Scale: the voxel index

Synthetic distractors padded the store to 10^4 , 10^5 , 10^6 (real query/target pairs, realistic perturbed-real distractors). Flat exhaustive search is the recall ceiling and the $O(n)$ baseline.

A single-level flat IVF (k-means Voronoi cells) at 128-d gave large speedups but **lost recall at scale**: at 1M, retention capped at 0.55 even probing 32 cells. This is high-dimensional distance concentration, the predicted curse, biting cell assignment.

Swapping to **HNSW** (the navigable-graph / octree analog) fixed it:

store	flat recall@10	HNSW best retention	speedup
10k	0.258	1.02	13x
100k	0.208	1.05	44x
1M	0.166	0.97	285x

HNSW holds full recall retention at a million items with a 285x scan-cost reduction. Dimensionality matters: HNSW at 32-d retained recall but at a *lower ceiling* (flat recall 0.072); at 128-d it recovered the ceiling (0.166) and kept retention. **128-d is the production dimension**, and it matches the fitted whitening output.

5. Cross-stack reach

The same primitive, “find the relevant few out of many in a space,” recurs throughout the platform:

- **Routing (already voxelized).** Meji/Odu *are* the coarse voxel levels of this hierarchy; formalizing them gives expert selection an explicit coarse-to-fine address with level-of-detail built in.
- **Oracle.** The 256-Odu deck is an 8-bit voxel grid over meaning; the TERA-biased draw is continuous-to-cell snapping.
- **Akashic Vault / AnythingLLM retrieval.** The HNSW bridge is generic; pointing it at document embeddings gives the Vault million-doc millisecond recall for free.
- **Training-corpus deduplication.** The separation probe found ~58% of spans have a near-twin. Voxel-consolidating the corpus before training removes redundant gradient and shrinks epochs, directly attacking on-device training cost. (Use a conservative threshold; some repetition aids learning.)
- **Swarm.** Voxelize the capability space and route a task to the nearest-capable peer, a geohash for experts.
- **Unity / DreamOS.** The world-building experts operate on literal 3-D space, where sparse voxel octrees are the original technique. The metaphor returns to reality in the engine.

6. Limitations and honest framing

- The recall numbers are a conservative lower bound: an adversarial cross-form, cross-domain test with no session scoping. The production loop is session-scoped and conversational, so operating recall is materially higher.
- The clean “100x” is not delivered by these levers alone in the worst case; it is reached through session scoping plus this stack, with a domain-tuned embedder as the remaining lever. We measured the path rather than asserting it.
- Whitening params are fit offline and reload on a store-fingerprint match; consolidation can drift a merged vector slightly between fits, which an approximate index tolerates.
- Resource contention was the binding constraint throughout: a 32 GB M1 Max running training with the corpora on an external volume, plus the full platform, starved concurrent em-

bedding jobs on I/O and CPU (not memory). Every experiment paid a resource tax, not an algorithmic one.

7. Reproducibility

The address hierarchy (`voxel.py`) is pure stdlib with a self-test. The memory loop, consolidation, whitening, and the HNSW bridge live in the running engine (`sidecar-placeholder.py`, `embedder_bridge.py`, `memory_index_bridge.py`). The separation probe, recall harness (with whitening/consolidation/reranker conditions), and the IVF/HNSW scaling sweeps are standalone scripts that cache embeddings so the sweeps re-run without re-embedding.

8. Conclusion

A single voxel idea unifies the memory architecture and reaches across the stack. The hierarchy was already present in AAMT’s Meji and Odu lattices; making it explicit, adding a learned residual, and indexing that residual with a navigable graph turns a fixed working set into a million-item sovereign Vault at constant attention cost, with full recall retention. The instinct that started it, “hypercubes reminded me of voxels,” was not a loose analogy. It was the missing structural layer, and it traces straight back to Minecraft chunks.